ГУАП

КАФЕДРА № 43

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

| старший преподаватель | | С.А. Рогачев |
|---|---|---|
| должность, уч. степень, звание | подпись, дата | инициалы, фамилия |

# ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №5

## Синтез КНА

по курсу: ТЕОРИЯ ВЫЧИСЛИТЕЛЬНЫХ ПРОЦЕССОВ

РАБОТУ ВЫПОЛНИЛ

| СТУДЕНТ ГР. № | 4236 | | А.В. Панченко |
|---|---|---|---|
| | | подпись, дата | инициалы, фамилия |

Санкт-Петербург 2025

## Цель работы:

Целью лабораторной работы является изучение алгоритма синтеза конечных недетерминированных автоматов (КНА) на основе регулярных выражений, построение КНА для заданного регулярного выражения с использованием данного алгоритма, а также создание программы на языке высокого уровня, реализующей этот алгоритм.

## Постановка задачи:

Построить КНА для регулярного выражения (c|d)<(b|c)|d>f, где <(b|c)|d> — опциональная часть (может быть или отсутствовать). Алфавит: {b, c, d, f}. Программа должна:

- Считывать регулярное выражение из файла input.txt.
- Считывать слово из файла input_text.txt.
- Записывать матрицу переходов и проверку слова в файл output.txt.
- Состояния нумеруются с 4 и последовательно до конечного.
- Представить автомат тремя способами, включая графическое описание в Mermaid.

## Алгоритм синтеза КНА:

Используется алгоритм Томпсона для построения NFA с ε-переходами:

- **Символ a:** NFA с переходом q_i --a--> q_f.
- **Конкатенация r1 r2:** Принимающее состояние r1 соединяется ε-переходом со стартовым r2.
- **Альтернатива r1 | r2:** Новое стартовое состояние с ε-переходами к стартовым r1 и r2; ε-переходы от принимающих r1 и r2 к новому принимающему.
- **Опциональность <r> (r?):</r>** Новое стартовое состояние с ε-переходами к стартовому r и новому принимающему; ε-переход от принимающего r к новому принимающему.

## Процесс синтеза КНА:

Регулярное выражение: (c|d)<(b|c)|d>f. Разбиваем:

- (c|d) — c или d.
- <(b|c)|d> — опциональная часть: (b|c) или d.
- f — финальный символ.

**Шаги построения:**

1. **Для (c|d):**
   - c: 0 --c--> 1
   - d: 2 --d--> 3
   - Альтернатива: 4 --ε--> 0, 4 --ε--> 2, 1 --ε--> 5, 3 --ε--> 5
   - Стартовое: 4, принимающее: 5
2. **Для (b|c):**
   - b: 6 --b--> 7
   - c: 8 --c--> 9
   - Альтернатива: 10 --ε--> 6, 10 --ε--> 8, 7 --ε--> 11, 9 --ε--> 11
   - Стартовое: 10, принимающее: 11
3. **Для d (внутри <(b|c)|d>):**
   - 12 --d--> 13
   - Стартовое: 12, принимающее: 13
4. **Для (b|c)|d:**
   - 14 --ε--> 10, 14 --ε--> 12, 11 --ε--> 15, 13 --ε--> 15
   - Стартовое: 14, принимающее: 15
5. **Опциональность <(b|c)|d>:**
   - 16 --ε--> 14, 16 --ε--> 17, 15 --ε--> 17
   - Стартовое: 16, принимающее: 17
6. **Конкатенация (c|d) с <(b|c)|d>:**
   - 5 --ε--> 16
   - Стартовое: 4, принимающее: 17
7. **Для f:**
   - 18 --f--> 19

- Стартовое: 18, принимающее: 19

8. **Конкатенация с f:**
    - 17 --ε--> 18
    - Стартовое: 4, принимающее: 19

**Итоговый NFA:** Стартовое состояние — 4, принимающее — 19, всего 20 состояний (0–19).

## Текст программы на языке высокого уровня:

```
import sys
from tabulate import tabulate

counter = 0

class NFA:
    def __init__(self):
        self.start = None
        self.accept = None
        self.transitions = {}
        self.epsilon = {}

def create_basic(sym):
    global counter
    s = counter
    counter += 1
    f = counter
    counter += 1
    nfa = NFA()
    nfa.start = s
    nfa.accept = f
    if (s, sym) not in nfa.transitions:
        nfa.transitions[(s, sym)] = set()
    nfa.transitions[(s, sym)].add(f)
    return nfa

def or_nfa(n1, n2):
    global counter
    s = counter
    counter += 1
    f = counter
    counter += 1
    nfa = NFA()
    nfa.start = s
    nfa.accept = f
    if s not in nfa.epsilon:
        nfa.epsilon[s] = set()
    nfa.epsilon[s].update({n1.start, n2.start})
    if n1.accept not in nfa.epsilon:
```

```
        nfa.epsilon[n1.accept] = set()
    nfa.epsilon[n1.accept].add(f)
    if n2.accept not in nfa.epsilon:
        nfa.epsilon[n2.accept] = set()
    nfa.epsilon[n2.accept].add(f)
    nfa.transitions = {**n1.transitions, **n2.transitions}
    nfa.epsilon = {**n1.epsilon, **n2.epsilon, **nfa.epsilon}
    return nfa

def concat_nfa(n1, n2):
    global counter
    nfa = NFA()
    nfa.start = n1.start
    nfa.accept = n2.accept
    nfa.transitions = {**n1.transitions, **n2.transitions}
    nfa.epsilon = {**n1.epsilon, **n2.epsilon}
    if n1.accept not in nfa.epsilon:
        nfa.epsilon[n1.accept] = set()
    nfa.epsilon[n1.accept].add(n2.start)
    return nfa

def optional_nfa(n):
    global counter
    s = counter
    counter += 1
    f = counter
    counter += 1
    nfa = NFA()
    nfa.start = s
    nfa.accept = f
    if s not in nfa.epsilon:
        nfa.epsilon[s] = set()
    nfa.epsilon[s].update({n.start, f})
    if n.accept not in nfa.epsilon:
        nfa.epsilon[n.accept] = set()
    nfa.epsilon[n.accept].add(f)
    nfa.transitions = n.transitions.copy()
    nfa.epsilon = {**n.epsilon, **nfa.epsilon}
    return nfa

class Parser:
    def __init__(self, s):
        self.s = s
        self.pos = 0

    def parse(self):
        nfa = self.parse_expr()
        if self.pos != len(self.s):
            print("Warning: Not all input parsed", file=sys.stderr)
        return nfa

    def parse_expr(self):
        nfa = self.parse_term()
```

```
        while self.pos < len(self.s) and self.s[self.pos] == '|':
            self.pos += 1
            n2 = self.parse_term()
            nfa = or_nfa(nfa, n2)
        return nfa

    def parse_term(self):
        nfa = self.parse_factor()
        while self.pos < len(self.s) and self.s[self.pos] not in '|)>':
            n2 = self.parse_factor()
            nfa = concat_nfa(nfa, n2)
        return nfa

    def parse_factor(self):
        if self.pos >= len(self.s):
            print("Error: Unexpected end of input", file=sys.stderr)
            return create_basic('')
        ch = self.s[self.pos]
        if ch == '(':
            self.pos += 1
            nfa = self.parse_expr()
            if self.pos < len(self.s) and self.s[self.pos] == ')':
                self.pos += 1
            else:
                print("Error: Missing )", file=sys.stderr)
            return nfa
        elif ch == '<':
            self.pos += 1
            nfa = self.parse_expr()
            if self.pos < len(self.s) and self.s[self.pos] == '>':
                self.pos += 1
            else:
                print("Error: Missing >", file=sys.stderr)
            return optional_nfa(nfa)
        else:
            self.pos += 1
            return create_basic(ch)

def epsilon_closure(states, epsilon):
    closure = set(states)
    stack = list(states)
    while stack:
        state = stack.pop()
        for next_state in epsilon.get(state, set()):
            if next_state not in closure:
                closure.add(next_state)
                stack.append(next_state)
    return closure

def move(states, symbol, transitions):
    next_states = set()
    for state in states:
        next_states.update(transitions.get((state, symbol), set()))
```

```
    return next_states

def accepts(nfa, word):
    current_states = epsilon_closure({nfa.start}, nfa.epsilon)
    for symbol in word:
        current_states = epsilon_closure(move(current_states, symbol,
nfa.transitions), nfa.epsilon)
        if not current_states:
            return False
    return nfa.accept in current_states

# Чтение регулярного выражения
try:
    with open('input.txt', 'r') as file:
        regex = file.read().strip()
except FileNotFoundError:
    print("Error: input.txt not found", file=sys.stderr)
    sys.exit(1)

# Чтение входного слова
try:
    with open('input_text.txt', 'r') as file:
        input_word = file.read().strip()
except FileNotFoundError:
    input_word = None
    print("Error: input_text.txt not found", file=sys.stderr)

counter = 0
parser = Parser(regex)
nfa = parser.parse()

# Формирование таблицы переходов
symbols = sorted(set(sym for _, sym in nfa.transitions))
table = []
headers = ["State"] + symbols + ["ε"]
for state in range(counter):
    row = [str(state)]
    for sym in symbols:
        nexts = nfa.transitions.get((state, sym), set())
        row.append(", ".join(map(str, sorted(nexts))) or "-")
    eps = nfa.epsilon.get(state, set())
    row.append(", ".join(map(str, sorted(eps))) or "-")
    table.append(row)

# Запись в файл output.txt
with open('output.txt', 'w') as file:
    file.write(f"Start state: {nfa.start}\n")
    file.write(f"Accept state: {nfa.accept}\n")
    file.write(f"Number of states: {counter}\n")
    file.write(f"Alphabet (without ε): {' '.join(symbols)}\n")
    file.write("\nTransition Table:\n")
    file.write(tabulate(table, headers=headers, tablefmt="grid"))
    if input_word is not None:
```

```
        file.write(f"\nInput string: {input_word}\n")
        file.write(f"Result: String '{input_word}' is {'accepted' if accepts(nfa,
input_word) else 'rejected'}\n")
    else:
        file.write("\nInput string: None (input_text.txt not found)\n")
```

## Текстовый файл выходных данных программы:

```
Start state: 4
Accept state: 19
Number of states: 20
Alphabet (without ε): b c d f
```

```
Input string: dcf
Result: String 'dcf' is accepted
```

Transition Table:

| State | b | c | d | f | ε |
|---|---|---|---|---|---|
| 0 | - | 1 | - | - | - |
| 1 | - | - | - | - | 5 |
| 2 | - | - | 3 | - | - |
| 3 | - | - | - | - | 5 |
| 4 | - | - | - | - | 0, 2 |
| 5 | - | - | - | - | 16 |
| 6 | 7 | - | - | - | - |
| 7 | - | - | - | - | 11 |
| 8 | - | 9 | - | - | - |
| 9 | - | - | - | - | 11 |
| 10 | - | - | - | - | 6, 8 |
| 11 | - | - | - | - | 15 |
| 12 | - | - | 13 | - | - |
| 13 | - | - | - | - | 15 |
| 14 | - | - | - | - | 10, 12 |
| 15 | - | - | - | - | 17 |
| 16 | - | - | - | - | 14, 17 |
| 17 | - | - | - | - | 18 |
| 18 | - | - | - | 19 | - |
| 19 | - | - | - | - | - |

## Конечный автомат заданный тремя способами:

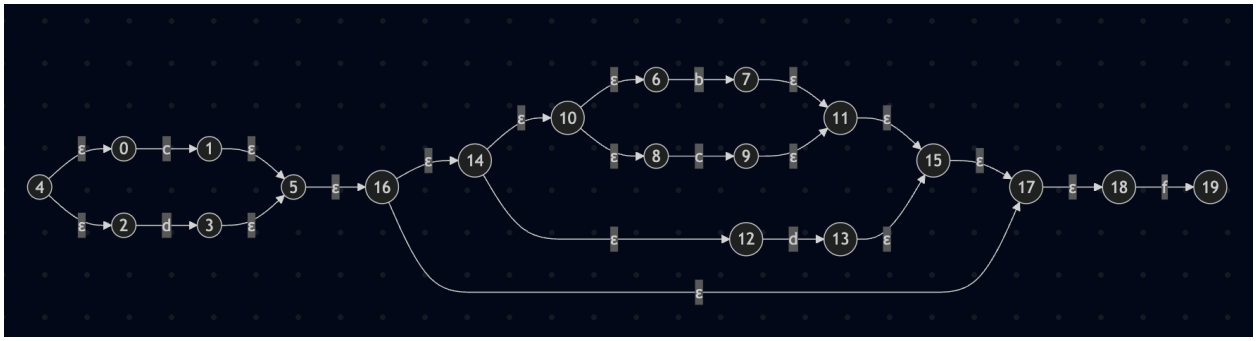
**Формальное описание:**

- M = (Q, Σ, δ, q0, F), где:
- Q = {0, 1, 2, ..., 19} (20 состояний),
- Σ = {b, c, d, f},
- q0 = 4 (стартовое состояние),
- F = {19} (принимающее состояние),
- δ — функция переходов, заданная в таблице выше.

**Таблица переходов**

См. содержимое output.txt выше — структурированная таблица с заголовками и границами.

**Графическое описание**




## Вывод:

В ходе лабораторной работы получено понимание регулярного выражения. Построен корректный NFA, который принимает строки вида cf, df, cbf, ccf, cdf, dbf, dcf, ddf (учитывая опциональность ). Программа на Python успешно синтезирует NFA и выводит матрицу переходов. Автомат представлен графически (Mermaid), в виде таблицы и формального описания, что соответствует требованиям задания.